

Why Async CUDA Streams Are a Small Lever: Anatomy of the `cudaMemcpy` Wait

Matias Saavedra

Wednesday 3rd June, 2026

Binary: `binary-v5-affinity` (fc33e29), HEAD eda2ae7 (docs only). **No new measurements.**

This report re-derives the cost of async CUDA streams from data already in `01-initial-profile.tex`, `09-9point-sampling.tex`, and `12-gpu-affinity.tex`, plus one code fact (the absence of `cudaSetDeviceFlags`) verified on fc33e29.

Resumen

This report revisits the asynchronous-CUDA-streams recommendation that five prior reports (01, 04, 05, 06, 07) carried as “the largest unexplored optimisation,” and that `09-9point-sampling.tex` began to walk back. No new run: it re-derives the cost from the CUDA-event and `nsys` data already collected, the per-region distribution of `12-gpu-affinity.tex`, and one code fact. The headline: the **13.4 ms** the API trace attributes to `cudaMemcpy` is **99.8 % kernel-wait** and only **~22 μ s** of actual device-to-host transfer, so async streams have essentially no transfer latency to hide. What they *could* reclaim is the GPU-driver thread’s stall — but that thread *busy-spins* (no `cudaSetDeviceFlags`), the stall is worth at most one **8.7 ms** median CPU region, two-way SMT on the 6-core/12-thread part discounts that to **~0.4** of a physical core, and the GPU is already **~91 %** utilised so the pipeline gap is under 9%. Net: a sub-1% wall lever, correctly ranked below work-reduction. The recommendation framing in reports 01/04 is retired.

1. Setup and Provenance

This is an analysis report, not a measurement campaign — in the same vein as `03-bug-analysis.tex` (pure code analysis) and `14-architecture-guide.tex` (no `experiments/` data). It produces no new numbers; it re-reads numbers already pinned to their own binaries and composes them into a single cost estimate for one optimisation that was never implemented. Every figure below is attributed to its source report.

Hardware.

Intel Core i7-9750H — **6 physical cores, 12 logical threads** (2-way SMT, “hyper-threading”) — plus an NVIDIA GeForce GTX 1660 Ti Max-Q. The renderer launches 12 worker threads by default (one per logical thread). *Thread 0 is the sole GPU-driver thread*; the other 11 are CPU-only. This 6/12 asymmetry is central to the argument and is why the single GPU-driver thread is not a free core.

Workload.

The canonical 100-frame zoom, 1920×1080 , `diffT` = 0.1, as in every prior report.

Data sources.

- `01-initial-profile.tex` (`binary-v0-buggy`): the original `nsys` API breakdown and the four-CUDA-event kernel/transfer split. Source of the 13.4 ms / 99 % / 47.4 s figures.
- `09-9point-sampling.tex` (`binary-v3-9point`): the cleanest `cudaMemcpy`-vs-transfer table (Table 1 here), and the ~ 95 ms total-transfer figure.
- `12-gpu-affinity.tex` (`binary-v5-affinity`, current best): the per-region distribution after GPU affinity (CPU median 8.7 ms) and the ~ 91 % GPU-utilisation ceiling.
- `kernel.cu` at `fc33e29`: the `hostFE` body (Listing below) and the verified absence of any `cudaSetDeviceFlags` call.

Limit on confidence.

Because nothing was re-run, the deltas quoted inherit the σ of their source reports; no new variance is claimed. The one quantity that is an *estimate* rather than a measurement — the SMT efficiency of the second logical lane — is flagged as such where it is used (`S3.3`).

2. Results

2.1. The code under discussion

The listing below is the relevant slice of `hostFE` in `kernel.cu`. The `mandelKernel` launch is asynchronous — it returns to the CPU immediately. The `cudaMemcpy` that follows runs on the *default* stream and is *blocking*: it cannot begin until `mandelKernel` has finished, so the calling thread (thread 0) parks inside it for the whole kernel runtime plus the transfer. The four `cudaEvent` stamps bracketing the two operations are precisely what let us separate kernel-wait from copy.

Código 1: `kernel.cu`, `hostFE()` – the launch/copy the API trace mis-attributes.

```
1 // Record a GPU timestamp immediately before the kernel launch.
2 CUDA_CHECK_RETURN(cudaEventRecord(evKernelStart));
3
4 mandelKernel<<<grid, block>>>(d_res, upperX, upperY, stepX, stepY,
5                               resX, resY, ptc, MAXITER); // async: returns at once
6
7 // Stamp the moment the kernel finishes (still async -- resolved below).
8 CUDA_CHECK_RETURN(cudaEventRecord(evKernelStop));
9 CUDA_CHECK_RETURN(cudaEventRecord(evMemcpyStart));
10
11 // Default-stream, BLOCKING copy: cannot start until mandelKernel finishes,
12 // so the GPU-driver thread parks here for the entire kernel runtime.
13 CUDA_CHECK_RETURN(
14     cudaMemcpy(h_res, d_res, resY * ptc, cudaMemcpyDeviceToHost));
15
16 CUDA_CHECK_RETURN(cudaEventRecord(evMemcpyStop));
```

```

17 CUDA_CHECK_RETURN(cudaEventSynchronize(evMemcpyStop));
18
19 // Two intervals: [start,stop] of the kernel, and [start,stop] of the copy.
20 float kernelMs = 0.0f, memcpyMs = 0.0f;
21 CUDA_CHECK_RETURN(cudaEventElapsedTime(&kernelMs, evKernelStart,
    ↪ evKernelStop));
22 CUDA_CHECK_RETURN(cudaEventElapsedTime(&memcpyMs, evMemcpyStart,
    ↪ evMemcpyStop));

```

2.2. Anatomy of one cudaMemcpy call

Tabla 1: What the `nsys` API trace attributes to `cudaMemcpy` versus what the four CUDA events show it actually is, over the 100-frame run (`binary-v3-9point`, report 09). The API-trace duration is **99.8 % kernel-wait**; the real device-to-host transfer is the last two rows. Read top-to-bottom: the first block is what a naive trace reader sees; the second is the truth the event timers expose.

Quantity	Value	Source
<i>What the API trace reports</i>		
<code>cudaMemcpy</code> share of CUDA API time	99.4 %	r09
<code>cudaMemcpy</code> avg per call	13.4 ms	r01
<code>cudaMemcpy</code> total over run	45.62 s	r09
<i>What the four events measure</i>		
real D → H transfer, avg per call	22.6 μs	r09
real D → H transfer, total over run	90.7 ms	r09
Derived kernel-wait share of cudaMemcpy	99.8 %	1 − 90.7 ms / 45.62 s

2.3. The marginal-work budget

Tabla 2: How much useful CPU work the GPU-driver thread’s per-region stall could buy back on the *current* binary, after each discount is applied in turn. Every row except the SMT efficiency is measured; that row is a stated estimate (see §3.3). The bottom line is the point of the report: a 13.4 ms stall is worth, at the physical-core level, well under half of one small region.

Step	Value	Source
GPU-driver stall per GPU region ($\approx$ kernel-wait)	13.4 ms	r01
CPU <i>median</i> region, GPU affinity	8.7 ms	r12
$\Rightarrow$ naive regions reclaimable / stall	~ 1.5	13.4/8.7
2nd-SMT-lane efficiency (sibling busy), <i>estimate</i>	$\sim 25\%$	assumed
$\Rightarrow$ net physical-core-equiv. reclaimed	~ 0.4 region	derived
Active worker threads / physical cores	12 / 6	hw
GPU utilisation (ceiling on pipeline gap)	$\sim 91\%$	r12
Thread 0 time spent in <code>cudaMemcpy</code> / wall	47.4 s / ~ 53 s	r01

3. Analysis

3.1. The reported 13.4 ms is 99.8% kernel-wait, not data transfer

Table 1: `nsys` attributes 99.4% of all CUDA API time and 13.4 ms per call to `cudaMemcpy`, yet the four CUDA events measure the actual device-to-host transfer at 22.6 μ s — a factor of ~ 590 smaller. The transfer is 90.7 ms over the entire 100-frame run; the API trace shows 45.62 s.

The mechanism is the call sequence in the listing above. A kernel launch is asynchronous: the `mandelKernel` launch queues the kernel and returns to the CPU at once. The very next blocking call, `cudaMemcpy` on the default stream, depends on the kernel’s output and the default stream is serialised, so it cannot start until the kernel completes. The CPU thread therefore sits *inside* `cudaMemcpy` for the entire kernel runtime and then the ~ 22 μ s copy. `nsys`’s API view has no way to know the thread spent that time waiting on the kernel rather than moving bytes — it charges the whole wall-clock span to the API call where the thread was parked. The reason the report-01 instrumentation exists at all is to break that span: `evKernelStart/Stop` bracket the compute, `evMemcpyStart/Stop` bracket the copy, and the difference is the artifact.

The implication is decisive for streams: async streams overlap *transfers* with compute. Here there are 90.7 ms of transfer across the run to overlap. Even perfect, zero-cost overlap of every transfer cannot recover more than 90.7 ms of a ~ 50 s wall — 0.18%. PCIe bandwidth was never the bottleneck.

3.2. That “wait” is a busy-spin, not a yield — the core is pinned, not freed

`kernel.cu` contains no `cudaSetDeviceFlags` call (verified at `fc33e29`), so the device runs under the default `cudaDeviceScheduleAuto` policy. Its documented heuristic spins when the number of active CUDA contexts is less than or equal to the number of logical processors, and yields otherwise. Here that is **1 context vs. 12 logical processors**, so the condition for *spinning* holds.

The mechanism, then, is that during the 13.4 ms kernel-wait, thread 0 does not sleep and hand its core back to the OS scheduler; it *busy-polls* the driver in a tight loop, keeping its logical lane near 100 % occupancy while producing no Mandelbrot pixels. This is the reason “the thread is just waiting” is subtly wrong: the thread is *spending* a CPU lane to wait, not releasing it. Were the policy `cudaDeviceScheduleBlockingSync`, the thread would block on an interrupt and free the lane — but that is not this binary.

The implication is that the resource async streams (or any restructuring) could reclaim is concrete and bounded: one busy-spinning logical lane, for 13.4 ms per GPU region. The next two subsections price it.

3.3. The reclaimable work is at most one median region — ~0.4 of a core after SMT

Table 2: on the current affinity binary the *median* CPU region is 8.7 ms (report 12), down from 88 ms under FIFO because GPU affinity vacuums the big coherent regions onto the GPU and leaves the CPU threads only small work. A 13.4 ms stall therefore spans ~1.5 median regions — naively, freeing thread 0 would let it finish about one-and-a-half small regions per GPU region.

Two discounts cut that down. **First, SMT.** Thread 0 is one of 12 logical lanes on 6 physical cores; its sibling lane already runs one of the other 11 CPU workers, so the physical core thread 0 sits on is *already computing pixels*. Reclaiming thread 0 does not add a core — it adds the marginal throughput of a second SMT lane on an already-busy core, which for compute-bound integer work of this kind is typically ~20–30 % (the estimate flagged in Table 2; not measured here). Applying ~25 % leaves ~0.4 region of genuinely new physical-core work per stall. **Second, the SMT knee.** Aggregated over the run, thread 0 spends 47.4 s of ~53 s inside `cudaMemcpy` (report 01) — it is already ~90 % a dedicated GPU feeder. Fully reclaiming it means moving the active worker count from 11 to 12 on 6 physical cores, where 11 threads already oversubscribe the cores 1.83×; the 12th lands on the flat part of the SMT scaling curve and contributes almost nothing.

The implication is that the per-stall prize is not “~1.5 regions” but “~0.4 of a physical core’s worth of a small region,” added past the SMT saturation point of a 6-core part — a fraction of a percent of CPU-pool throughput.

3.4. The GPU is ~91 % utilised, so there is almost no gap to pipeline into

Report 12 measured the GPU at ~91 % utilisation under affinity (95–96 % on the heavy frames of report 13). A software pipeline — launch region $n+1$ ’s kernel while region n ’s copy/host-work proceeds — only helps to the extent the GPU has idle gaps

between kernels to fill.

The mechanism is straightforward: async streams convert serial `launch` $\rightarrow$ `wait` $\rightarrow$ `launch` into overlapped issue, recovering the inter-kernel idle. At $\sim 91\%$ utilisation that idle is $\leq 9\%$ of GPU time by construction; affinity (report 12) already spent most of the available scheduling headroom getting there. There is little left for a pipeline to claim.

The implication is that the two independent ceilings agree: the transfer to hide is 0.18% of wall (S4.1), and the GPU idle to fill is $< 9\%$ of an already-saturated device — and the CPU-side beneficiary of either is a ~ 0.4 -core lane added past the SMT knee. All three bound the same lever to sub-1% of wall.

3.5. Why five prior reports overstated it

Reports 01, 04, 05, 06 and 07 each listed async streams as a top-tier lever; 04 called it “still the largest unexplored optimisation ... recover most of that 13 ms.” The reason is visible in Table 1: read literally, the `nsys` API trace puts 47.4 s — nearly the whole wall — inside `cudaMemcpy`, which looks like an enormous, obviously-overlappable transfer cost. The overstatement is exactly the kernel-wait artifact of S4.1 propagating into a recommendation.

09-9point-sampling.tex was the first to correct it in print, noting the transfer “is only ~ 95 ms, so async streams recover GPU-thread *idle* around the kernel, not transfer time.” This report completes that correction by pricing the idle itself (S4.2–4.4): it is a busy-spun lane worth ~ 0.4 of a physical core, against a $< 9\%$ GPU gap. The implication is that the ranking should be made explicit and the stale framing retired.

4. Recommendations

1. **Keep async streams below work-reduction in priority.** The combined ceiling from S4.1–4.4 is sub-1% of wall (0.18% of transfer to hide; $< 9\%$ GPU idle whose CPU-side payoff is a ~ 0.4 -core lane). Work-reduction — periodicity checking in `diverge()` to skip iterations on in-set pixels — attacks the ~ 595 s of *conserved* total compute that actually bounds the wall (report 09). Trade-off: streams also demand a non-trivial restructure (per-region double-buffering, deferred completion across the GPU thread’s region lifetime) for that sub-1% return.
2. **If the spin cost must be confirmed before any investment, do the one-line A/B first.** Add `cudaSetDeviceFlags(cudaDeviceScheduleBlockingSync)` once in `CUDAmemSetup` and re-run the 100-frame wall. This frees thread 0’s lane to the OS during each kernel-wait instead of spinning; the predicted result, from S4.3, is *within noise* — the freed lane lands past the SMT knee on the 6-core part. Cost: 1 line, 1 run. A null result here is positive evidence that the streams restructure would also be null.
3. **Retire the “largest unexplored optimisation” framing.** The recommendation bullets in reports 01, 04, 05 and 06 predate the cost decomposition above; this report supersedes them. The async-streams entry in `CLAUDE.md`’s “Recommended next steps” already ranks it lower-priority; this is the quantitative

backing for that ranking.

5. Conclusion

The 13.4 ms that the `nsys` API trace charges to `cudaMemcpy` is 99.8 % kernel-wait and ~ 22 μ s of real transfer — a measurement artifact of timing a blocking copy that follows an asynchronous kernel launch on the default stream. Async CUDA streams hide transfer latency, of which this workload has 90.7 ms total (0.18 % of wall). The only other thing they could reclaim — the GPU-driver thread’s stall — is a busy-spun logical lane worth at most one 8.7 ms median region per GPU region, discounted by two-way SMT on the 6-core/12-thread part to ~ 0.4 of a physical core and added past the SMT saturation knee, while the GPU it feeds is already ~ 91 % utilised. Three independent ceilings put the same lever under 1 % of wall. What was carried through five reports as “the largest unexplored optimisation” is, once the kernel-wait is separated from the transfer, one of the smallest. The binding constraint remains the ~ 595 s of conserved compute; the next lever is work-reduction, not overlap.